

# Oefententamen 2; Programmeren, AI en Samenleving

Naam:

.....

## Inloggen:

- Je maakt dit tentamen in de online editor. Ga naar: **progbg.proglab.nl/exams**.
- Voer de tentamencode in:
- Wacht tot de docent je studentenkaart of ID heeft gecontroleerd.
- De docent voert vervolgens jouw persoonlijke **PIN** in.
- Terwijl je wacht, lees je alvast deze tentameninstructies verder door.

## Doel:

- Dit digitale tentamen bestaat uit 6 opdrachten waarin je korte Python-programma's schrijft.
- Je hebt minimaal 3 correcte opdrachten nodig om het tentamen te halen.
- Je wordt **alleen beoordeeld op de correctheid** van je oplossingen. Code-design, commentaar en stijl zijn niet van belang, dus daar hoeft je geen zorgen over te maken.
- De editor heeft een Checkpy-knop waarmee je tijdens het tentamen je oplossingen kunt controleren voor eigen gebruik. Let op: de Checkpy-resultaten hebben geen invloed op je uiteindelijke cijfer, want het nakijken gebeurt handmatig. Dus ook als je oplossingen niet slagen voor Checkpy, kunnen ze alsnog goedgekeurd worden bij de beoordeling.

## Randvoorwaarden:

- Je mag de cursuswebsite als bron gebruiken (open [progbg.proglab.nl](http://progbg.proglab.nl) opnieuw in een nieuw tabblad nadat je bent ingelogd).
- Je mag een geprinte versie van de cheatsheet gebruiken tijdens het tentamen.
- Gebruik geen externe modules zoals `numpy`, `csv` of andere modules, tenzij de opdracht expliciet aangeeft dat het mag.

## Inleveren Wanneer je alle opdrachten hebt afgerond:

- gebruik de **submit-knop** op de website om ervoor te zorgen dat je laatste wijzigingen zijn opgeslagen,
- **vul alsjeblieft de evaluatie in** (link in het submit-venster), dit helpt ons enorm om de cursus te verbeteren,
- sluit de browser,
- lever dit tentamenblad in **met je naam erop**.

## Opdracht 1: Fop

Schrijf een functie genaamd `fop(n)`. Deze functie retourneert een lijst met alle *fop*-getallen tussen 1 en `n` (inclusief 1, exclusief `n` zelf). We definiëren een *fop*-getal als een getal `n`, zodat  $n \cdot n + 1$  deelbaar is door 5. Bijvoorbeeld 3 is een *fop*-getal want  $3 \cdot 3 + 1 = 10$  en 10 is deelbaar door 5.

### Voorbeeld

```
fopl = fop(10)
print(fopl)
```

```
fopl = fop(35)
print(fopl)
```

### Verwachte Output

```
[2, 3, 7, 8]
[2, 3, 7, 8, 12, 13, 17, 18, 22, 23, 27, 28, 32, 33]
```

Tips: Je kunt de `%`-operator gebruiken om te testen of een getal deelbaar is.

## Opdracht 2: Twitterize

We vonden Twitter toch echt een beter naam dan X. Schrijf een functie, `twitterize(text)`. Het doel van de functie is een nieuwe string te produceren gebaseerd op `text` naar overal waar in de input `text` het woord 'X' staat, bevat de output het woord 'Twitter'. Doe dit alleen voor de hoofdletter 'X', niet de kleine letter 'x', en zorg ervoor dat 'X' niet wordt vervangen als het een onderdeel is van een ander woord (zoals 'Xylofoon').

### Voorbeeld

```
text = """X is a silly name for a company.
Try to search X in google, it's 10X harder to find than Twitter."""
twitterized = twitterize(text)
print(twitterized)
```

### Verwachte Output

```
Twitter is a silly name for a company.
Try to search Twitter in google, it's 10X harder to find than Twitter.
```

## Opdracht 3: Dot

Schrijf een functie `dot(l1, l2)` die twee lijsten (`l1` en `l2`) van gelijke lengte als invoer accepteert. Beide lijsten bevatten getallen. De functie moet het volgende doen:

1. Vermenigvuldig elk element in `l1` met het overeenkomstige element in `l2`. Dit betekent dat het eerste element van `l1` wordt vermenigvuldigd met het eerste element van `l2`, het tweede element van `l1` met het tweede element van `l2`, enzovoort.
2. Bereken de som van alle berekende producten.

Als voorbeeld: stel dat de invoer `[1, 2, 4]` en `[5, 2, 3]` is. De functie berekent dan:  $1 \cdot 5 + 2 \cdot 2 + 4 \cdot 3 = 21$ . De functie retournt dus in dit geval 21.

### Voorbeeld 1

```
d = dot([1, 2, 4], [5, 2, 3])
print(d)
```

### Verwachte Output

21

### Voorbeeld 2

```
d = dot([10, 20, 30, 40], [1, 1, 0, -1])
print(d)
```

### Verwachte Output

-10

## Opdracht 4: Palindroom

Schrijf een functie `find_palindromes(text)` die alle palindroomwoorden in een opgegeven tekst (`text`) vindt. Een palindroom is een woord dat hetzelfde blijft wanneer het van achteren naar voren wordt gelezen. Bij het bepalen of een woord een palindroom is, moeten leestekens worden genegeerd en moet er geen onderscheid worden gemaakt tussen hoofdletters en kleine letters.

De functie retournt een lijst met alle palindroomwoorden in de volgorde waarin ze in de tekst voorkomen. Zorg ervoor dat de woorden in lijst wel dezelfde hoofd- en kleine letters bevatten als in de input tekst, maar geen leestekens.

### Voorbeeld

```
text = "A kayak is a deified boat. A Honda Civic is not a racecar."
result = find_palindromes(text)
print(result)
```

### Verwachte Output

```
['A', 'kayak', 'a', 'deified', 'A', 'Civic', 'a', 'racecar']
```

## Opdracht 5: Scrabble

Deze opdracht is gebaseerd op het bordspel scrabble. Schrijf een functie, `scrabble_options(word_list, letters)` die bepaalt welke woorden je allemaal kan vormen met een geven lijst met `letters`. De mogelijke woorden waar je uit kan kiezen wordt gegeven `word_list`. De functie retournt de *gesorteerde* lijst met woorden die je kan vormen met de gegeven `letters`.

### Voorbeeld

```
# full list of words in IDE
english_words = ['alike', 'apple', 'beach', 'beard', 'bicycle', 'bread', ...
my_letters =  ["a", "a", "b", "r", "p", "d", "e", "p", "l", "z", "s", "t"]
print(scrabble_options(english_words, my_letters))
```

### Verwachte Output

```
['apple', 'beard', 'bread', 'plaza', 'salad', 'table', 'zebra']
```

## Opdracht 6: Uncaesar

Wat staat hier: "mttwfd, dtz ini ny! ltti otg. :)"?

Deze tekst is versleuteld met een Caesar-versleuteling. Bij deze versleuteling wordt elke letter een aantal (in dit geval 5) plaatsen opgeschoven in het alfabet. Dus, de 'a' -> 'f', de 'b' -> 'g', enzovoort. Aan het einde van het alfabet wordt er "rondom" gegaan, zodat 'y' -> 'd' en 'z' -> 'e'. Non-alfabetische karakters worden niet versleuteld. Schrijf een functie `uncaesar(text, n)`, die een tekst (`text`) die met `n` plaatsen versleuteld is, weer ontcijfert.

Je mag aannemen dat de input-tekst allen maar kleine letters bevat.

### Voorbeeld

```
encoded = "mttwfd, dtz ini ny! ltti otg. :)"
decoded = uncaesar(encoded, 5)
print(decoded)
```

### Verwachte Output

???

(Zet je oplossing in het commentaar onder je oplossing.)

### Tip

Als je wilt achterhalen waar een letter (bijvoorbeeld 'p') in het alfabet staat kan je deze constructie gebruiken:

```
index_p = "abcdefghijklmnopqrstuvwxyz".index("p") # a -> 0, z -> 25
```